Bitcoin Is Not a Hard Drive

Josh · Secure Sovereign

Published July 31, 2026 · Updated September 20, 2026

The Technical and Philosophical Case Against Non-Monetary Data Embedding

Contents

- I. What Bitcoin Actually Is
- II. The Type Confusion
- III. The Three Costs
- IV. The Externality Structure
- V. Forced Participation
- VI. The Justifications and Their Failures
- VII. The Permissionless Counter-Argument
- VIII. What Can Actually Be Done
- Conclusion

I. What Bitcoin Actually Is

Bitcoin is a distributed consensus state transition ledger. Every node maintains an identical copy **not because storing data is valuable, but because reaching agreement about who owns what is valuable.**

The system was built for **one purpose: settling monetary transactions without a trusted third party.** To do that, every participant must maintain and validate the complete chain of prior state transitions. The integrity of the current state depends on the integrity of every state before it. That requirement is not optional infrastructure around the money. **It is the mechanism itself.**

What Bitcoin can technically be made to carry and **what Bitcoin was built to carry** are different things. That gap is the problem this document addresses.

II. The Type Confusion

In software engineering, **type confusion** occurs when a value of one kind is passed to a context designed for something fundamentally different. The bytes are accepted, but the system was not built for them.

Bitcoin's transaction fields exist because payments require them. Hash fields exist for hash-then-reveal privacy, so a recipient can withhold a public key until spend time in case cryptographic assumptions weaken. Amount fields exist for satoshi precision. Sequence numbers and locktimes exist for transaction replacement and timelocked contracts. These fields are **not a general-purpose data bus with a monetary use case bolted on.** They are payment infrastructure that happens to be technically capable of carrying other things.

`OP_RETURN` was introduced as a controlled exit valve for provably unspendable outputs, ones that cannot be mistaken for monetary UTXOs and do not need to be tracked as potential future spends. Peter Todd proposed it on bitcoin-dev on 2013-02-12 as the honest unspendable marker. Jeff Garzik, on 2014-02-24, described the 0.9 change that made a small metadata output standard as damage control against MasterCoin-style unspendable outputs bloating the UTXO set. He rejected the reading that Core developers were endorsing data storage. Eighty bytes because hashes plus a tag. Luke Dashjr, on 2014-11-16, treated relay size as miner policy, not a Core development topic.

Consensus never capped the field. The current default is `MAX_OP_RETURN_RELAY = MAX_STANDARD_TX_WEIGHT / WITNESS_SCALE_FACTOR`, which is 100,000 vbytes, with `DEFAULT_ACCEPT_DATACARRIER` true. The original relay limit was friction that kept the channel aligned with its purpose, marking outputs as definitively unspendable without turning them into subsidized bulk storage.

The **Taproot envelope** is an `OP_FALSE OP_IF` branch that never executes. It was designed as an upgrade hook, not a data container. SegWit's witness discount exists because signature data, while necessary for validation, does not contribute to the UTXO set burden that every node must track indefinitely. Applying that discount to image files treats a targeted engineering accommodation as a general feature.

Using any of these fields to carry arbitrary non-monetary data is type confusion: the bytes are accepted, and the costs land on infrastructure built for something else.

The pattern is not unique to Bitcoin. A relational database is optimized for normalized, queryable data with defined relationships. Storing blob data in it anyway is technically possible, but indexing, querying, replication, and backup all absorb costs the schema was never built to handle. Early VoIP ran over TCP because TCP was available, not because it was appropriate. TCP's retransmission guarantee is a feature for file transfer and a liability for real-time audio, because a retransmitted packet arrives too late to play and the protocol has no way to discard it. Storing millions of small files in a filesystem built for a modest number of large ones causes inode exhaustion, directory traversal collapse, and backup failures. In each case the system accepts the input and the mismatch shows up as hidden costs across everything that depends on the layer below.

That is **impedance mismatch at the abstraction boundary.** Every layer assumes something about what the layer below will carry. Violating the assumption does not erase the mismatch; it spreads cost across every operation that touches that layer. Bitcoin is worse than most of these cases because the others allow migration. You can extract blob data from SQL, refactor trigger logic, move to UDP for media. **Bitcoin's chain history is immutable.** No later engineering decision undoes costs already imposed.

III. The Three Costs

One megabyte of payment data and one megabyte of arbitrary data are not equal in network impact. They differ across three cost categories.

Initial block download. When a new participant joins the network, they download and validate the entire chain history from the genesis block forward. Every non-monetary payload ever embedded passes through that node's validation pipeline. **Spam does not become free once it is old.** It becomes a permanent toll on every future participant who wants to verify their own transactions.

Non-monetary data accounts for an estimated **12 to 19 percent** of total chain storage. Blockspace devoted to spam intensified roughly **17-fold** relative to its pre-inscription baseline. A node operator syncing today pays for all of it, permanently. For the full operator-cost model (including the ~$5.50/month non-monetary slice inside ~$70/month operating cost), see *Full Cost of Running a Bitcoin Node*.

Pruning does not escape this. A pruned node still downloads and validates the full chain history during initial sync. What pruning removes is the ongoing storage obligation afterward, at the cost of surrendering the ability to re-verify the full chain from the node's own copy. A pruned node must trust an external node for that function, sacrificing sovereignty.

Ongoing storage. A full archival node stores the complete chain indefinitely. The chain currently requires approximately **700 gigabytes** of storage and grows with every block. Arbitrary data embedding inflates that number directly. Every non-monetary byte embedded today is a byte every archival node carries forever, with no recovery mechanism.

UTXO set bloat. The UTXO set is the live working set of all unspent transaction outputs. Every validating node holds it in fast storage to check whether incoming transactions are spending real outputs. Its size directly affects validation performance.

Payment UTXOs have economic agents who have reasons to spend them. When a payment output is spent, it leaves the UTXO set.

Inscription-related outputs represent approximately **29.6 percent** of all UTXOs while holding around **415 bitcoin** in total value. The ratio of monetary weight to chain footprint inverts completely compared to payment outputs. Almost no one has economic reason to spend them, so they accumulate in the UTXO set without cycling out, permanently occupying fast storage on every validating node on the network.

For measured blockspace impact and channel-by-channel cost tables, see *The Achievable Floor, §VI*.

IV. The Externality Structure

These three cost categories share a common structure. **The person who embedded the data paid a fee once.** Miners were compensated at the moment of inclusion. **Every node operator pays** storage, bandwidth, and UTXO set costs for the entire remaining lifetime of the chain, with no compensation, no recovery path, and no relationship to the person who imposed the costs.

The fee mechanism does what it was designed to do: compensate miners for including transactions in blocks. It was **not designed to compensate node operators** for carrying those transactions indefinitely. That gap reflects the original design assumption that transactions would serve monetary purposes, and that monetary purposes would create economic agents with reasons to eventually resolve the state they created.

Arbitrary data embedding severs that relationship. The minting cost is paid once. The carrying cost is distributed permanently across everyone who validates the network, including everyone who joins later and had no role in the original decision. A transaction that imposes permanent costs on unpaid third parties with no recovery path is not paying its own way. **The fee covers inclusion, not the externality.**

V. Forced Participation

The **"just don't run a node"** response misunderstands what nodes are for.

Bitcoin's security model depends on broad node participation. The properties that distinguish Bitcoin from a database someone else controls (verifying your own transactions, confirming your own balance, rejecting invalid blocks without trusting anyone) all require running a validating node. Custodial services and lite clients inherit their security from the nodes they trust. If the cost of running a node rises beyond what individuals can absorb, the validating population shrinks toward institutions and the network's decentralization degrades.

Policy filters and mempool settings do not resolve this. A transaction with sufficient fee reaches a miner regardless of whether other nodes will relay it. Direct submission APIs, alternative relay networks, and private pool peering all exist and are actively used. Once a transaction is in a valid

block, every node must accept it. **Consensus validity is what binds.** A node operator's mempool policy is not a vote on what enters the chain. Participating in Bitcoin's security model means paying all the costs that consensus validity imposes, including those from arbitrary data embedding.

For the structural logic of why social enforcement cannot substitute for consensus closure here, see *The Social Layer Is the Attack Surface* and *Argument Map, Part VI*.

VI. The Justifications and Their Failures

Five defenses recur. Each borrows a real property of Bitcoin and treats the use case as if it inherits that property, without pricing the cost imposed on the infrastructure those properties depend on.

Immutability and censorship resistance. The claim is that storing data on Bitcoin makes it permanent and uncensorable. Bitcoin's immutability was built to protect **one specific thing: the finality of monetary transactions.** It guarantees that a confirmed transfer of value cannot be reversed or erased. It was not built as a general archival service. Treating court-record reliability as a reason to store personal correspondence there makes the same mistake. A court is reliable for court records, not for every extra payload someone attaches. The attachment still costs everyone who needs the court for its job.

The same permanence guarantee is available without putting the data on Bitcoin. Arweave was designed for permanent, censorship-resistant storage with a one-time payment that funds retention. IPFS provides content-addressed storage identified by a cryptographic hash of the contents, so tampering is detectable without a central authority. The working pattern is to **store the data on IPFS or Arweave and commit only the content hash on chain**, anchoring provenance and timestamp. The data lives on infrastructure built to carry it. You get Bitcoin-anchored proof of existence without putting the file on Bitcoin.

Fee payment as legitimacy. The argument is that paying the fee makes any use legitimate and the market has spoken. Fees compensate miners at inclusion. They do not compensate node operators for the permanent costs above. The fee mechanism was calibrated for monetary transactions whose state has an economic lifecycle. Paying the inclusion fee for a use case that externalizes permanent costs onto unpaid third parties is **not the market pricing a service.** It is offloading costs the market price does not capture onto people who have no way to decline.

Miner fee revenue and the security budget. This is the strongest fee-market argument. Inscriptions generated substantial miner fee revenue, and as block rewards decline, fee revenue matters for security. But miner revenue and node operator participation run on **different timescales.** Miner revenue from inscription fees is real and immediate. Erosion of the node operator population from rising participation costs is also real, and it compounds over years. A

network with high miner revenue and a shrinking validating population is not more secure. It is more dependent on trusting miners, which is the attack surface node decentralization exists to close. Paying one layer while shifting costs onto another does not improve the security model.

Innovation and new users. The argument is that inscriptions attract developers and communities who would otherwise ignore Bitcoin. People attracted by cheap data storage are not the same as people attracted by sound money, and they have no stake in node decentralization. Using Bitcoin as a data store is not innovative. Purpose-built decentralized storage (IPFS, Filecoin, Arweave) has been mature for years. Choosing Bitcoin instead is **a preference for subsidized infrastructure** over paying the real cost of what you want, with the subsidy taken from node operators who never agreed to provide it.

The block weight limit as containment. The claim is that Bitcoin's roughly four-million weight-unit block ceiling already proves spam is tolerable: damage has an upper bound, so the network has already contained it. The weight limit keeps **one block** within what a node can validate in finite time. It does not decide which uses of that scarce capacity belong in a monetary security model, and it does not pay node operators for the permanent stock of non-monetary bytes that accumulate under that rate.

A fire code that caps burn rate does not make arson acceptable. **Bounded rate is not the same as acceptable purpose.** The limit is the shared scarce resource payments need. Filling it with arbitrary data means the "upper bound on damage" is also an upper bound on monetary settlement capacity. That is the problem, not the defense.

The witness discount weakens the containment claim further: the same weight units admit more raw envelope payload than equivalent non-witness payment data, so the ceiling is asymmetrically generous to the dedicated spam channel. UTXO set bloat is not cured by a per-block byte ceiling either; outputs that never need to be spent stay in every node's fast storage indefinitely. For rate-versus-stock accounting, BIP141 asymmetry, and measured fullness, see *The Achievable Floor, §VI*.

VII. The Permissionless Counter-Argument

The strongest objection is that Bitcoin is permissionless, and any mechanism that excludes a class of transactions is a step toward excluding any transaction. If the network can decide that JPEGs do not belong, it can decide that dissidents do not belong.

That collapses a distinction that matters. **Bitcoin's permissionless property applies to monetary transactions.** It means no authority can prevent you from sending value to another party. It does **not** mean Bitcoin is a neutral substrate for any purpose that can be technically encoded. Those are different claims.

The original `OP_RETURN` limit was not censorship of monetary transactions. It was friction that kept a non-payment channel from becoming a subsidized data bus at the expense of node operators who chose to validate monetary settlement. It was removed under documented pressure from actors with direct financial interest in using Bitcoin's infrastructure as cheap data storage. The governance outcome and the financial interests behind it are both matters of public record. For the full documented account, see *Who Controls Bitcoin, §V*, the *Argument Map, Parts VI–VII and XXII*, and *What Bitcoin's Stalled Proposals Tell You* (policy monoculture section).

Treating every technical constraint as equivalent to political censorship makes it impossible to distinguish between a limit on arbitrary data embedding and a limit on sending bitcoin to a dissident. The permissionless property is meaningful precisely because it applies to something specific, and extending it to cover every possible use of every transaction field (compounding costs on every node that validates the network) degrades the very thing it claims to defend.

VIII. What Can Actually Be Done

Arbitrary data cannot be reduced to zero. The fields that carry embedded data overlap with fields that payments require, and closing those fields would destroy payment functionality.

Hash fields carrying fake pubkeys or script hashes, amount fields encoding data in low-order bits, and sequence numbers and locktimes within valid ranges all resist consensus closure because the same fields serve legitimate payment functions. Distinguishing a real pubkey hash from a fake one at the consensus layer is not possible without stripping hash-then-reveal privacy from every payment.

What consensus **can** close are the **dedicated high-bandwidth channels:** `OP_RETURN` outputs with large payloads, and the Taproot envelope used as a bulk data container. These channels serve no monetary function, and closing them removes nothing that payments require. That closure is specified in *Permanent Data Channel Closure*.

Closing dedicated channels does not eliminate arbitrary data embedding. It forces embedders into hash and pubkey output fields that carry a dust floor on each output and cost more per embedded byte at every fee rate. The goal is making non-monetary use costly and unwelcome rather than subsidized by infrastructure built for something else. For the full technical breakdown of which channels consensus can close and where the floor actually sits, see *The Achievable Floor*.

UTXO commitments address the historical storage burden. A node holding a root hash and verifying spends through inclusion proofs supplied by spenders does not need to carry the full UTXO set locally. The research has been complete since 2014. Every node that syncs today still pays the full cost of chain history that commitments would have capped a decade ago, and that cost grows with every new participant.

A **consensus-enforced per-output miner fee** addresses UTXO set bloat going forward. Two tools are easy to confuse. A minimum output value floor locks bitcoin in each output; that capital comes back when the output is spent, so a spammer can recycle it. A per-output miner fee is different: every newly created output pays a fixed fee to the miner, as ordinary transaction fee, that never comes back. That makes each spam run permanently more expensive, with no recycled capital to fund the next one. Inputs spend outputs created earlier, so if new outputs carry a permanent fee at consensus, stuffing data into later spend fields still requires paying that creation cost up front. Dedicated-channel caps and a per-output fee are complementary, and neither requires closing payment-necessary fields. See *Static Per-Output Miner Fee* and the optional anti-decay companion *Dynamic Escalation of the Per-Output Miner Fee*. The three pre-proposals are one stack: the fee BIPs price the UTXO slot; the data-channel BIP closes the bus.

Conclusion

Bitcoin's value as a monetary network depends on the cost of running a validating node staying within reach of individuals. The moment that cost is accessible only to institutions, the security model becomes dependent on trusting those institutions, and the property that distinguishes Bitcoin from everything that came before it begins to dissolve.

Arbitrary data embedding raises that cost across three dimensions: **the one-time download every future participant pays at sync**, **the indefinite storage every archival node carries**, and **the permanent UTXO set burden every validating node holds in fast storage.** None of these costs are compensated by the fee that covered inclusion, and all of them fall on people who had no vote on whether the data belonged there.

Monkey JPEGs are beside the point. The question is whether the people running the infrastructure that makes Bitcoin work should subsidize other people's data storage, permanently and without exit, because payment fields can technically be made to carry other things.

The externalities are not free. Not every valid byte is equally aligned with the security model. Calling that neutrality is a choice about who pays.

Bitcoin is not a hard drive.

Companion to The Achievable Floor (channel taxonomy), Full Cost of Running a Bitcoin Node (operator cost model), Why Shitcoins Are Shitcoins (altcoin monetary-asset case), Who Controls Bitcoin (governance evidence), Bitcoin Governance: Argument Map (numbered arguments), and the three-BIP stack: Permanent Data Channel Closure, Static Per-Output Miner Fee, Dynamic Escalation of the Per-Output Miner Fee.

SECURESOVEREIGN

Markdown is the canonical source: https://secsov.com/articles/bitcoin-not-a-hard-drive/index.md

Formatted snapshot of https://secsov.com/articles/bitcoin-not-a-hard-drive

Josh · Secure Sovereign